

《面向对象程序设计实践》

课程论文

题目：	电子图片管理程序
专业：	计算机科学与技术/软件工程
班级：	24 级 5 班
小组成员 1：	202425220519 龙绍宇
2：	202425220508 邓杰
3：	202425220510 方宇
指导老师：	彭红星
提交时间：	2026 年 5 月 19 日

华南农业大学 数学与信息学院 软件学院

目 录

✧ 1 系统分析.....	3
✧ 2 系统设计.....	5
✧ 3 系统实现.....	8
✧ 4 系统测试.....	13
✧ 5 系统运行界面.....	15
✧ 6 总结.....	17

1 系统分析

1.1 问题描述

随着手机、数码相机以及网络图片资源的普及，普通用户电脑中保存的图片数量明显增加。图片一多以后，单纯依靠系统文件夹进行查找和整理就不太方便，特别是在需要快速预览、批量选择、批量复制、删除或重命名时，操作过程比较零散，也容易出现遗漏。

本项目设计并实现一个 Java 电子图片管理程序，目标不是做一个特别复杂的商业软件，而是完成一个能真正运行、逻辑清楚、交互直观的桌面应用。程序以本地文件系统为基础，左侧显示磁盘与文件夹目录，右侧显示当前目录下的图片缩略图，并提供图片选择、删除、复制、粘贴、重命名以及幻灯片播放等功能。说得简单一点，就是让用户不用一张张打开图片，也能比较顺手地完成日常整理。

从课程设计角度看，该项目覆盖了面向对象程序设计中的类封装、对象协作、事件监听、文件读写、图形界面布局和异常处理等内容。相比只写命令行程序，这个项目更接近实际应用，因为用户的每一次点击、双击、拖动和右键操作都需要程序及时响应。

1.2 需求分析

根据课程题目要求，系统需要使用 Java 图形用户界面技术完成程序界面，能够显示 JPG、JPEG、GIF、PNG、BMP 等常见图片格式，并且能够处理几十 KB 到几 MB 不同大小的图片。项目功能主要分为两个部分：一是图片预览部分，二是幻灯片播放部分。

编号	需求名称	具体说明
R1	目录浏览	只显示磁盘和文件夹结构，不在目录树中显示普通文件。
R2	图片预览	以缩略图方式显示当前目录中的图片，并保持原图比例。
R3	图片选择	支持单选、Ctrl 多选和鼠标拖动框选，选中状态需要有明显区别。
R4	图片管理	支持删除、复制、粘贴和重命名，删除前需要确认，粘贴重名时需要自动处理。
R5	幻灯片播放	支持上一张、下一张、放大、缩小和自动播放。
R6	扩展功能	在基本要求上增加图片左转、右转和水平翻转功能。

1.3 系统功能分析

结合源代码实现情况，系统功能可以进一步细分为目录树加载、图片筛选、缩略图显示、图片选中、文件操作、图片变换和幻灯片播放几个模块。各模块之间不是孤立的，比如目录树点击后会触发图片加载，图片加载后才可以进行选择和右键菜单操作，右键菜单处理完成后又需要重新刷新图片预览区域。

程序主界面由 MainFrame 负责组织，图片缩略图由 ImageLabel 单独封装，幻灯片播放窗口由 SlideShowFrame 处理，图片读取、缩放、旋转和翻转则由 ImageUtils 提供工具方法。这样的划分比较符合面向对象思想，也让代码看起来不至于全部堆在一个类里面。

在功能体验上，系统既实现了课程要求中的基础功能，也增加了比较实用的图片方向调整功能。比如有些图片从手机导入电脑后方向不正确，用户可以直接在预览界面右键旋转，也可以在幻灯片播放窗口中边看边调整，处理完成后直接覆盖保存原文件。

1.4 开发平台及工具介绍

项目	工具或技术	说明
开发语言	Java	用于编写桌面应用主体逻辑。
开发工具	IntelliJ IDEA	用于代码编写、项目管理与运行调试。
界面技术	Java Swing	使用 JFrame、JTree、JPanel、JLabel、JScrollPane、JPopupMenu、JOptionPane 等组件。
文件处理	java.io.File、 java.nio.file.Files	完成目录读取、图片筛选、复制、删除和重命名等操作。
图片处理	ImageIO、 BufferedImage、 Graphics2D、 AffineTransform	完成图片读取、缩放、旋转、翻转与保存。
运行环境	Windows + JDK	不依赖数据库和网络服务，本地即可运行。

本项目没有使用数据库，也没有使用外部服务器。图片数据直接来自用户电脑中的文件夹，程序操作的对象就是本地图片文件。这样的设计比较轻量，部署成本低，符合课程设计的规模。

1.5 可行性分析

从技术可行性看，项目使用的知识点主要是 Java 基础语法、Swing 事件模型、文件系统操作和二维图像处理。虽然 Swing 写界面时细节比较多，但只要把组件布局、事件监听和数据刷新关系理清，整体实现难度是可控的。

从操作可行性看，系统采用左侧目录树、右侧缩略图、底部提示信息的布局方式，用户打开程序后基本不用额外学习。点击目录就能看图，点击图片就能选中，右键就能管理，双击或按钮就能进入幻灯片播放，这些交互方式和普通桌面软件比较接近。

从维护可行性看，程序已经将主要职责分配到不同类中。后续如果继续扩展，如增加图片排序、图片信息查看、按日期分组或搜索功能，可以在现有结构上继续补充，不需要推翻整体架构。

2 系统设计

2.1 系统总体结构设计

系统总体上采用“主窗口 + 图片项组件 + 播放窗口 + 图片工具类”的结构。主窗口负责调度，图片项组件负责缩略图显示，播放窗口负责大图查看，工具类负责通用图像算法。

程序启动时，Main 类通过 SwingUtilities.invokeLater 进入 Swing 事件派发线程，并设置系统默认外观，然后创建 MainFrame 主窗口。主窗口初始化目录树、图片面板和右键菜单，之后所有用户操作都围绕这些组件展开。

流程	说明
步骤 1	程序启动 Main.main，进入 Swing 事件派发线程。
步骤 2	创建 MainFrame，初始化目录树、图片预览区域和底部提示信息。
步骤 3	用户在目录树中选择文件夹，系统筛选图片文件并生成 ImageLabel 缩略图组件。
步骤 4	用户进行单选、多选、框选或右键菜单操作，MainFrame 调用对应业务方法。
步骤 5	双击图片或点击播放按钮，创建 SlideShowFrame 进行大图查看和自动播放。
步骤 6	图片缩放、旋转和翻转等公共操作统一交给 ImageUtils。

图 2-1 系统总体运行流程说明

2.2 系统各类及类之间关系设计

类名	角色	主要职责
Main	程序入口类	设置 Swing 外观并创建主窗口。
MainFrame	主界面类	继承 JFrame，负责目录树、图片预览、右键菜单、文件操作和播放入口。
ImageLabel	缩略图组件类	继承 JPanel，封装图片文件、缩略图、文件名和选中状态。
SlideShowFrame	幻灯片窗口类	继承 JFrame，负责大图显示、上一张/下一张、缩放和自动播放。
ImageUtils	图片工具类	负责图片读取、等比例缩放、左转、右转、水平翻转和保存。

MainFrame 和 SlideShowFrame 都需要处理图片显示，但二者的关注点不同。MainFrame 更侧重目录管理和批量操作，SlideShowFrame 更侧重单张大图的连续查看。ImageUtils 则被两个窗口共同调用，避免旋转、缩放等代码重复出现。ImageLabel 的设计比较小，但作用很明显。每个缩略图不仅仅是一张图片，还包含文件对象、文件名显示和选中状态。主窗口在处理复制、删除、重命名时，通过 ImageLabel 就可以找到对应的 File 对象，这样界面元素和实际文件之间建立了直接联系。

2.3 数据存储设计

本系统没有设计数据库，全部数据均存放在本地文件系统中。程序运行时通过 File 对象引用磁盘目录和图片文件，图片本身仍然保存在原来的文件夹中。复制、删除、重命名、旋转和翻转操作会直接作用于本地文件。

数据名称	类型	作用
------	----	----

nowFolder	File	保存当前正在浏览的目录。
picList	List<ImageLabel>	保存当前目录下已经显示的缩略图组件。
copyList	List<File>	保存执行复制操作后等待粘贴的图片文件。
selected	boolean	保存在 ImageLabel 中，用于表示该缩略图是否被选中。
nowIndex	int	保存在 SlideShowFrame 中，用于记录当前播放到第几张图片。
scale	double	保存在 SlideShowFrame 中，用于记录当前图片缩放比例。

文件命名方面，粘贴操作会检测目标目录是否已有同名文件，如果存在同名文件，则自动生成“_copy1”“_copy2”这类后缀，避免覆盖用户原有图片。批量重命名时，程序先把原文件改成临时文件，再改成最终名称，这样可以减少同目录下文件名互相冲突的问题。

2.4 界面设计

主界面采用左右分栏结构。左侧是目录树区域，用 JTree 显示磁盘和文件夹；右侧是图片预览区域，用 FlowLayout 排列多个 ImageLabel 缩略图；底部是提示信息区域，用 JLabel 显示当前目录、图片总数、总大小和选中数量。

右键菜单包含复制、粘贴、删除、重命名、向左旋转 90 度、向右旋转 90 度、水平翻转等功能项。用户选中图片后，在选中图片上点击右键即可弹出菜单。这里把常用操作放在右键菜单中，使用起来比较贴近文件管理器的习惯。

幻灯片窗口的设计更简单，中间大区域显示当前图片，下方按钮栏提供上一张、下一张、放大、缩小、左转、右转、水平翻转和播放按钮。考虑到图片可能放大到超过窗口大小，程序在图片区域外层放置 JScrollPane，使用户可以滚动查看。

2.5 交互与异常处理设计

系统的交互设计尽量遵循“操作前确认、操作后反馈”的原则。删除图片前会弹出确认框；复制成功、粘贴成功、删除成功和批量重命名成功后，底部提示信息会显示处理结果；如果用户没有先选中图片就执行操作，程序会用 JOptionPane 给出提示。

图片读取、文件复制、图片保存等操作都有可能失败，例如文件损坏、权限不足或路径发生变化。源代码中对 IOException、NumberFormatException 等异常进行了捕获，并通过提示框或空图标方式处理，避免程序因为单个文件错误直接崩溃。

3 系统实现

3.1 程序启动与主窗口构建

程序入口位于 Main 类。为了保证 Swing 组件在正确线程中创建，代码使用 `SwingUtilities.invokeLater` 启动界面。这一点虽然只有几行代码，但对 Swing 程序来说是比较规范的写法。

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) {}
    });
    new MainFrame().setVisible(true);
}
```

MainFrame 构造方法中设置窗口标题、大小、关闭方式和整体布局，然后调用 `initTree`、`initPicPanel`、`initMenu` 三个方法分别初始化目录树、图片预览区域和右键菜单。主窗口使用 `JSplitPane` 将左侧目录树和右侧预览区分开，底部通过 `JLabel` 显示提示信息。

3.2 目录树加载与图片筛选实现

目录树使用 `File.listFiles()` 获取系统磁盘根目录。为了避免程序一启动就扫描所有目录，代码采用了“懒加载”的方式：如果某个目录下面还有子文件夹，就先放入一个“正在加载...”占位节点，只有当用户展开该节点时，才真正读取其子目录。这个处理让程序启动速度更快，也减少了无意义的文件系统访问。

```
private DefaultMutableTreeNode makeNode(File folder) {
    DefaultMutableTreeNode node = new DefaultMutableTreeNode(folder);
    if (hasFolder(folder)) {
        node.add(new DefaultMutableTreeNode(TEMP_NODE));
    }
    return node;
}

private boolean hasFolder(File folder) {
    File[] arr = folder.listFiles(file -> file.isDirectory() && !file.isHidden());
    return arr != null && arr.length > 0;
}
```

当用户点击目录树中的某个目录后，`clickTree` 方法会取得当前节点对应的 `File` 对象，然后调用 `showPictures` 方法刷新右侧图片区域。图片筛选通过扩展名完成，只保留 `jpg`、`jpeg`、`gif`、`png`、`bmp` 格式，并过滤隐藏文件和非文件对象。

```
private boolean isPic(File folder, String name) {
    File file = new File(folder, name);
    if (!file.isFile() || file.isHidden()) {
        return false;
    }
    String s = name.toLowerCase();
    return s.endsWith(".jpg") || s.endsWith(".jpeg")
}
```

```

        || s.endsWith(".gif") || s.endsWith(".png")
        || s.endsWith(".bmp");
    }

```

3.3 缩略图显示与选中状态实现

缩略图显示由 `ImageUtils.getScaledIcon` 实现。程序先读取原始图片宽高，再根据目标显示区域计算缩放比例，取宽高比例中的较小值作为最终比例。这样可以保证图片按照原始比例缩小，不会出现被拉宽或压扁的情况。

```

double r1 = (double) w / oldW;
double r2 = (double) h / oldH;
double r = Math.min(r1, r2);
if (r > 1.0) {
    r = 1.0;
}
int newW = Math.max(1, (int) Math.round(oldW * r));
int newH = Math.max(1, (int) Math.round(oldH * r));
Image newImg = img.getScaledInstance(newW, newH, Image.SCALE_SMOOTH);

```

每张缩略图由 `ImageLabel` 表示。`ImageLabel` 内部保存 `File` 对象、图片显示标签、文件名标签和 `selected` 变量。当 `selected` 为 `true` 时，组件背景变为浅蓝色并显示蓝色边框；未选中时显示白色背景和灰色边框。这个效果不复杂，但用户一眼就能看出当前选中了哪些图片。

```

private void changeBorder() {
    if (selected) {
        setBackground(new Color(220, 235, 255));
        setBorder(BorderFactory.createLineBorder(new Color(30, 100, 210), 2));
    } else {
        setBackground(Color.WHITE);
        setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(230, 230, 230)),
            BorderFactory.createEmptyBorder(1, 1, 1, 1)));
    }
    repaint();
}

```

3.4 单选、多选和框选实现

图片选择功能由鼠标事件实现。单击图片时，如果没有按 `Ctrl` 键，程序会先清空其他图片的选中状态，再选中当前图片；如果按住 `Ctrl` 键，则只切换当前图片的选中状态，从而实现多选。双击图片时，程序会直接打开幻灯片播放窗口，并从双击的图片开始播放。

```

if (e.getClickCount() == 2) {
    openPlay(picList.indexOf(label));
    return;
}
if (e.isControlDown()) {
    label.setSelected(!label.isSelected());
} else {
    clearChoose();
    label.setSelected(true);
}
updateTip();

```


框选功能写在 MainFrame 的内部类 MyPanel 中。程序记录鼠标按下点和拖动点，根据两点生成矩形区域，然后判断该矩形是否与每个缩略图组件的边界相交。如果相交，就把对应图片设为选中。实际用起来，这个功能对批量选择图片很方便。

```
private void chooseByRect(Rectangle r, boolean addMode) {
    if (!addMode) {
        clearChoose();
    }
    for (ImageLabel label : picList) {
        if (r.intersects(label.getBounds())) {
            label.setSelected(true);
        }
    }
    updateTip();
}
```

3.5 复制、粘贴、删除与重命名实现

复制功能并不是立即复制文件，而是把当前选中的图片文件保存到 copyList 中。等用户进入目标目录并点击粘贴时，程序再使用 Files.copy 将这些图片复制到当前目录。这样用户可以先选择图片，再切换目录进行粘贴，符合一般文件管理习惯。

```
private void pastePic() {
    if (nowFolder == null) {
        JOptionPane.showMessageDialog(this, "请先选择目标文件夹。", "提示",
        JOptionPane.INFORMATION_MESSAGE);
        return;
    }
    int ok = 0;
    for (File oldFile : copyList) {
        File newFile = getNewFile(nowFolder, oldFile.getName());
        Files.copy(oldFile.toPath(), newFile.toPath(),
        StandardCopyOption.REPLACE_EXISTING);
        ok++;
    }
    showPictures(nowFolder);
    tipLabel.setText("提示: 成功粘贴 " + ok + " 张图片。");
}
```

删除功能在真正删除文件前会弹出确认对话框。只有用户确认以后，程序才遍历选中的 ImageLabel 并调用 File.delete 删除对应文件。删除后调用 showPictures 重新加载当前目录，保证界面显示和磁盘文件状态一致。

重命名分为单张重命名和批量重命名。单张重命名时用户直接输入新的主文件名，扩展名保持不变；批量重命名时，用户输入名称前缀、起始编号和编号位数，程序按文件名排序后依次生成新文件名。为了避免多个文件之间因为名称变化产生冲突，批量重命名时先改成临时名，再改成最终名。

```
String newName = pre.trim() + String.format("%0" + width + "d", start + i) + ext;
File newFile = new File(oldFile.getParentFile(), newName);

File middle = new File(oldFile.getParentFile(), oldFile.getName() + ".tmp" +
System.nanoTime());
```

```
oldFile.renameTo(middle);
// 全部改成临时名后，再统一改成最终文件名。
```

3.6 图片旋转与水平翻转实现

本项目在基本要求之外增加了图片左转 90 度、右转 90 度和水平翻转功能。主窗口中，这三个功能放在右键菜单里；幻灯片播放窗口中，这三个功能以按钮形式出现。具体图像处理代码集中在 ImageUtils 中，便于两个窗口复用。

以右转 90 度为例，程序先读取原图，创建宽高互换的新 BufferedImage，然后通过 Graphics2D 平移并旋转坐标系，最后把原图绘制到新图像中并写回原文件。这里要注意，JPG 和 BMP 不支持透明通道，所以保存时需要转换为 RGB 类型，否则可能出现保存失败或背景异常。

```
public static boolean rotateRight(File file) {
    BufferedImage img = readImage(file);
    int w = img.getWidth();
    int h = img.getHeight();
    BufferedImage newImg = new BufferedImage(h, w, img.getType());
    Graphics2D g = newImg.createGraphics();
    g.translate(h, 0);
    g.rotate(Math.toRadians(90));
    g.drawImage(img, 0, 0, null);
    g.dispose();
    return saveImage(newImg, file);
}
```

水平翻转使用 AffineTransform 完成。程序先把 x 方向缩放设置为 -1，再把图像平移回可见区域，相当于把图片左右镜像。处理成功后，主界面会重新刷新缩略图，播放窗口也会重新读取当前图片，用户能马上看到处理结果。

```
AffineTransform at = new AffineTransform();
at.scale(-1, 1);
at.translate(-w, 0);
g.drawImage(img, at, null);
```

3.7 幻灯片播放窗口实现

幻灯片播放窗口由 SlideShowFrame 实现。MainFrame 打开播放窗口时，会把当前目录中的图片文件列表传入，并传入起始下标。如果用户双击某张图片，就从该图片开始播放；如果点击“进入幻灯片播放”按钮，则从第一张图片开始播放。

上一张和下一张功能由 nowIndex 控制。每次切换图片时，程序把 scale 恢复为 1.0，然后重新读取并显示图片。如果已经到第一张或最后一张，程序会给出提示。自动播放功能使用 Swing Timer，每隔 1 秒调用 nextPic(false) 切换到下一张。

```
private void autoPlay() {
    if (timer == null) {
        timer = new Timer(1000, e -> nextPic(false));
    }
    if (timer.isRunning()) {
```

```
        stopPlay();
    } else {
        timer.start();
        playBtn.setText("停止");
    }
}
```

3.8 界面刷新与提示信息实现

程序在完成文件操作后，通常会调用 `showPictures(nowFolder)` 重新加载当前目录。虽然这种方式不是性能最高的做法，但逻辑简单、结果可靠。图片数量不是特别大时，重新刷新界面不会有明显卡顿。

底部提示信息由 `updateTip` 方法统一维护。该方法会统计当前目录中图片总数、总大小和已选中数量，并显示在界面底部。这样用户每次选择图片后，都能及时知道当前操作状态。

```
tipLabel.setText("提示: 当前目录 " + name + ", 共有 " + picList.size()
    + " 张图片, 总大小 " + getSizeText(total)
    + ", 已选中 " + chooseCount + " 张。");
```

4 系统测试

4.1 测试环境

本项目在 Windows 环境下进行测试，开发工具为 IntelliJ IDEA，运行环境为 JDK。测试时准备了多个文件夹和不同格式的图片，包括 JPG、JPEG、PNG、GIF 和 BMP，也准备了同名图片、空文件夹、图片数量较多的文件夹等情况，用来检查程序在常见场景下是否稳定。

测试项	说明
操作系统	Windows 10/Windows 11
开发工具	IntelliJ IDEA
运行环境	JDK 8 及以上版本
测试图片格式	JPG、JPEG、PNG、GIF、BMP
测试数据规模	单目录 0 张、1 张、多张以及几十张图片

4.2 模块测试

编号	模块	输入/操作	预期结果	实际结果	结论
M01	目录树加载	启动程序并展开磁盘目录	左侧显示文件夹结构，不显示普通文件	与预期一致	通过
M02	图片筛选	选择包含多种文件的目录	右侧只显示支持格式图片	与预期一致	通过
M03	缩略图显示	打开含大图和小图的目录	缩略图按比例显示，不变形	与预期一致	通过
M04	单张选择	鼠标左键点击一张缩略图	该图片显示选中边框，提示选中 1 张	与预期一致	通过
M05	Ctrl 多选	按住 Ctrl 连续点击多张图片	多张图片保持选中状态	与预期一致	通过
M06	拖动框选	在预览区域拖动形成矩形	矩形范围内图片被选中	与预期一致	通过

4.3 系统功能测试

编号	功能	输入/操作	预期结果	实际结果	结论
S01	删除图片	选中多张图片后点击右键删除	弹出确认框，确认后文件删除并刷新	正常删除并刷新	通过
S02	复制粘贴	复制图片后切换目录粘贴	图片复制到目标目录，界面更新	正常复制	通过
S03	同名粘贴	在同一目录中粘贴同名图片	自动生成 _copy 后缀，不覆盖原文件	生成新文件名	通过
S04	单张重命名	选择 1 张图片并输入新文件名	主文件名改变，扩展名不变	符合预期	通过
S05	批量重命名	多选图片，输入前缀、起始编号和位数	按顺序生成编号文件名	符合预期	通过
S06	旋转翻转	对图片执行左转、右转、水平翻转	图片方向改变并保存到原文件	符合预期	通过
S07	幻灯片切换	双击图片进入播放窗口并切换	可查看上一张和下一张，边界有提示	符合预期	通过
S08	自动播放	点击播放按钮	每隔约 1 秒自动切换	符合预期	通过

			图片，可停止		
--	--	--	--------	--	--

4.4 测试结果分析

从测试结果看，系统已经能够完成课程设计要求中的主要功能。目录树能够正确加载本地文件夹，图片预览区域能够筛选并显示支持格式的图片，单选、多选和框选操作也能够正常更新选中状态。

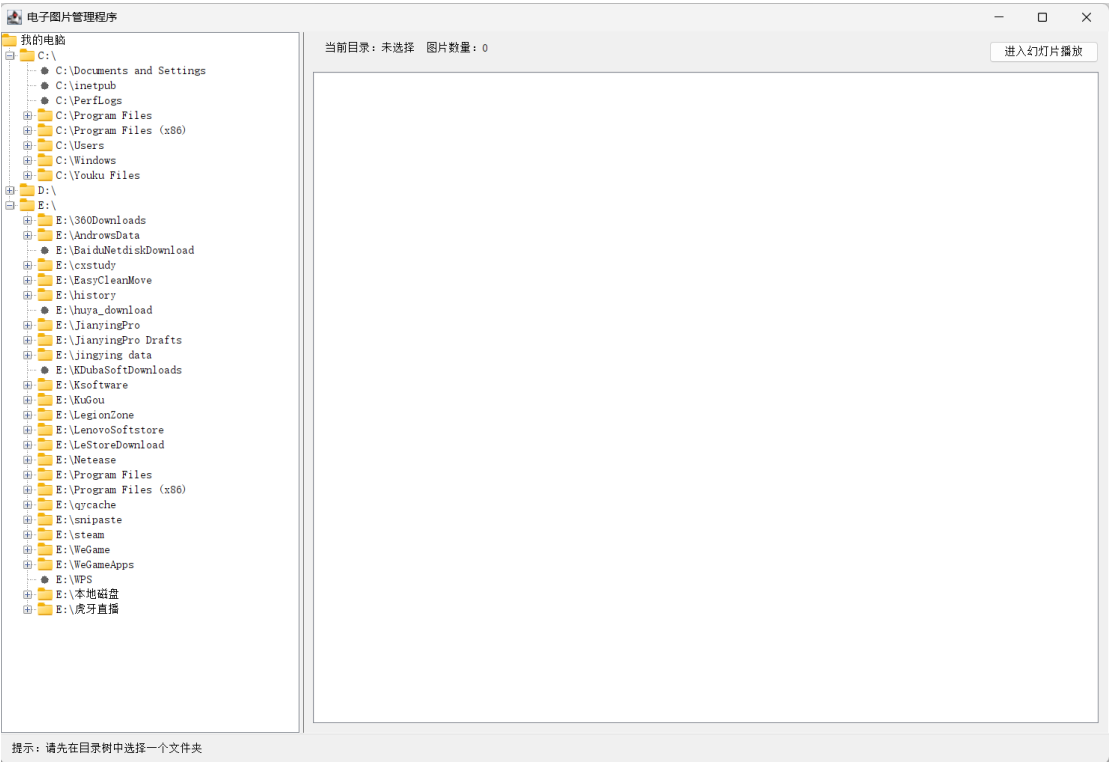
文件管理部分整体表现正常。复制粘贴能够处理同名文件，删除操作有确认提示，单张重命名和批量重命名都保留了原扩展名。图片旋转和水平翻转能够覆盖保存原文件，处理完成后界面刷新及时，用户不需要手动重新进入目录。

幻灯片播放部分可以从双击图片进入，也可以从主界面按钮进入。上一张、下一张、放大、缩小和自动播放功能均能正常使用。播放窗口中加入旋转和翻转按钮以后，查看图片和调整图片方向可以放在同一个窗口中完成，使用体验比只在主界面操作更方便。

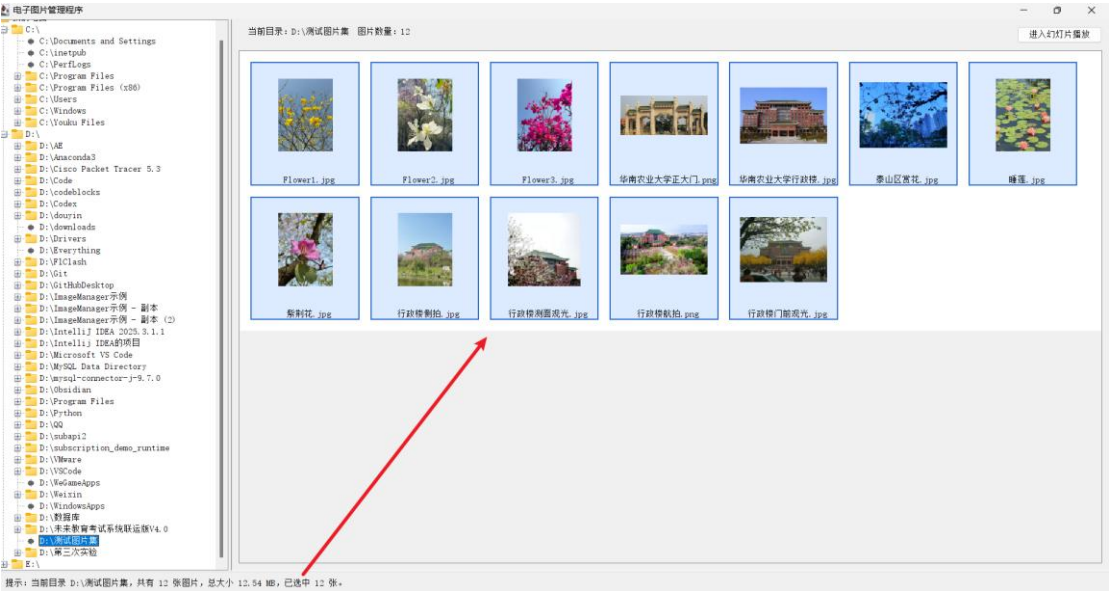
当然，测试中也能看出一些后续可改进的地方。例如，当单个目录中图片数量特别多时，全部重新加载缩略图可能会变慢；旋转 GIF 动图时可能只处理第一帧；删除操作目前是直接删除文件，没有放入回收站。

5 系统运行界面

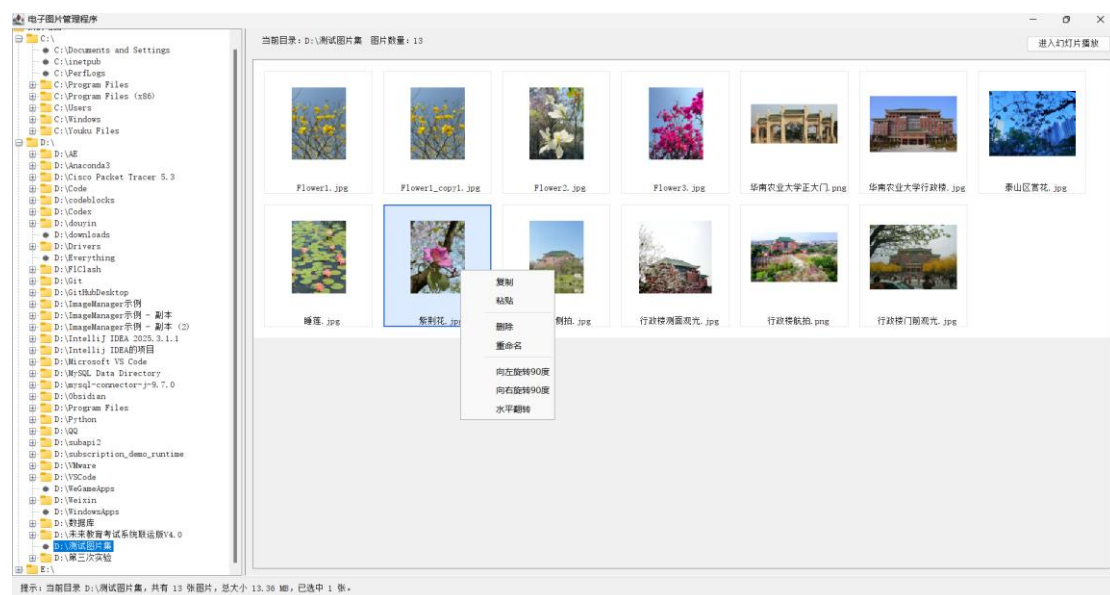
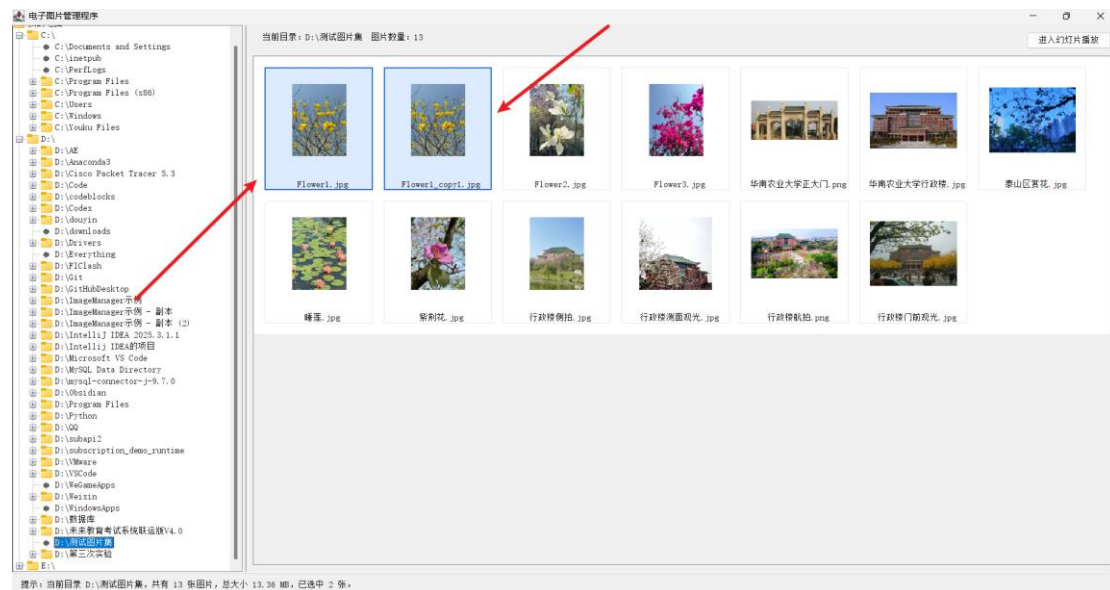
主界面截图



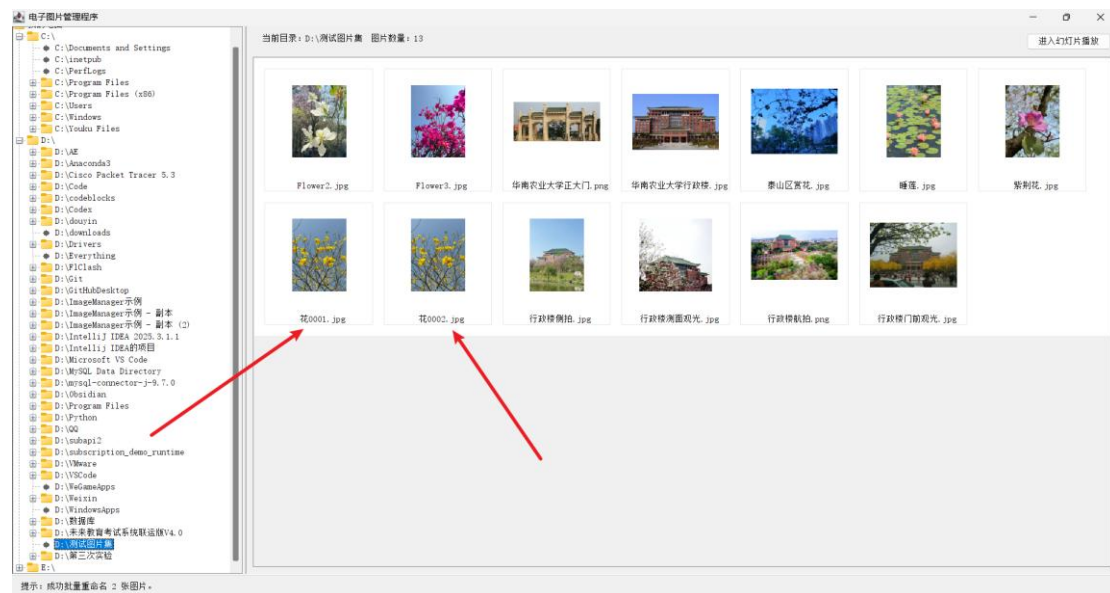
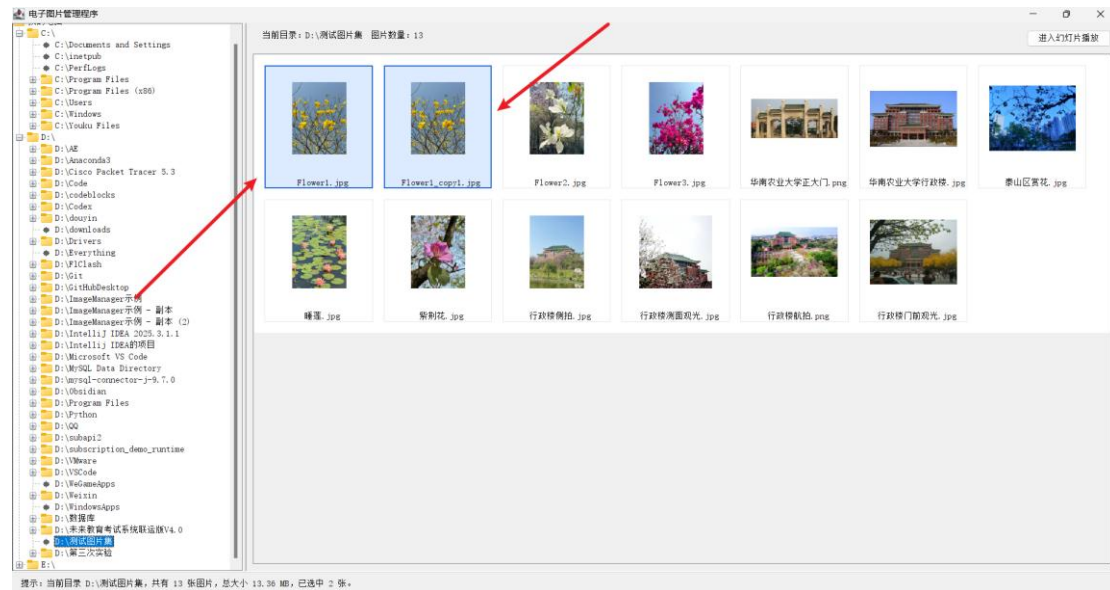
多张图片选择截图



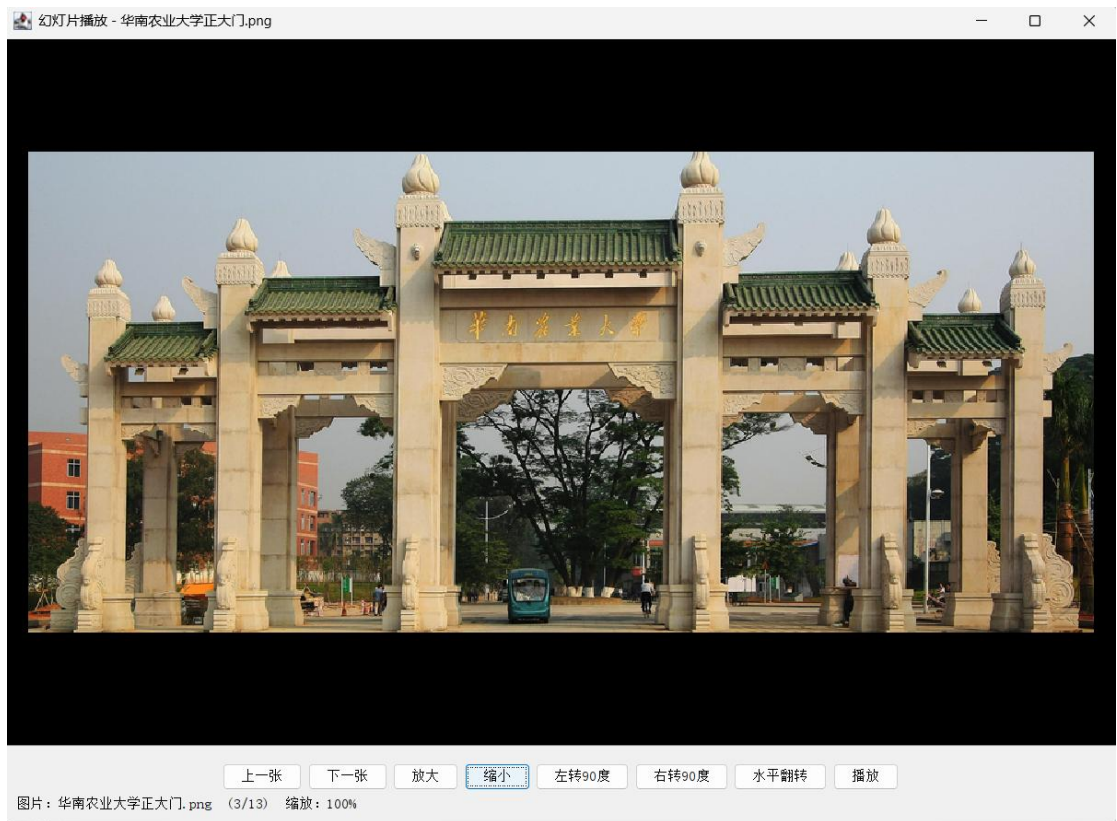
右键菜单操作截图



重命名与批量重命名截图



幻灯片播放窗口截图



6 总结

6.1 项目完成情况总结

通过本次 Java 电子图片管理程序的设计与实现，项目基本完成了课程题目中要求的图片预览和幻灯片播放两大部分功能。程序能够浏览本地目录，显示多种格式图片，支持单选、多选、框选、删除、复制、粘贴、重命名、放大缩小和自动播放，同时还增加了图片旋转与水平翻转功能。

从代码结构上看，项目不是简单把所有代码写在一个 main 方法里，而是划分为 MainFrame、ImageLabel、SlideShowFrame 和 ImageUtils 等类。虽然还不能说架构特别完善，但已经体现了面向对象程序设计中“分工明确、职责相对独立”的思想。

在实现过程中，比较有收获的部分是 Swing 事件处理和图片处理。以前写按钮点击事件时可能只关注一个功能点，这次需要考虑目录树点击、缩略图点击、双击、右键、拖动框选、播放按钮和键盘快捷键之间的配合，才发现图形界面程序的细节其实很多。

6.2 小组成员 1 总结

小组成员 1：龙绍宇

本次课程设计中，我主要参与了系统需求分析、主界面功能设计和核心代码整理。通过这个项目，我对 Java Swing 的窗口布局、事件监听和组件刷新有了更直接的理解。特别是在图片选择和文件操作部分，代码不仅要能运行，还要考虑用户误操作、文件重名、目录为空等情况。做完以后感觉收获比较实在，也认识到桌面程序看起来简单，真正写完整并不轻松。

6.3 小组成员 2 总结

小组成员 2：邓杰

本次课程设计中，我主要参与了图片预览、缩略图显示和幻灯片播放相关功能的分析与测试。项目让我进一步熟悉了 File、ImageIO、BufferedImage 和 Graphics2D 等类的使用。图片缩放时要保持比例，播放窗口中还要处理放大、缩小和自动切换，这些细节让我对图像处理 and 界面交互之间的关系有了更清楚的认识。

6.4 小组成员 3 总结

小组成员 3：方宇

本次课程设计中，我主要参与了文件管理功能和系统测试部分。删除、复制、粘贴和重命名看上去都是常见操作，但真正实现时需要处理确认提示、同名文件、扩展名保留和批量编号等问题。通过测试不同目录和不同格式的图片，我也体会到测试并不是简单点几下按钮，而是要根据功能设计不同场景，观察预期结果和实际结果是否一致。

6.5 后续改进方向

如果后续继续完善该系统，可以考虑增加图片排序方式，如按名称、大小、修改时间排序；也可以增加图片详细信息查看功能，例如分辨率、文件大小、创建时间等。除此之外，还可以加入搜索框，让用户快速查找当前目录中的图片。

在性能方面，可以考虑对缩略图进行缓存，减少反复读取大图片带来的卡顿；在文件安全方面，删除图片时可以改为移动到回收站，而不是直接调用 `File.delete`；在界面方面，也可以进一步优化按钮样式和布局，让程序看起来更接近成熟软件。总体来说，本项目完成了课程设计的核心目标，也为以后继续学习 Java 图形界面开发打下了基础。